

SIMATS ENGINEERING

ASSESSMENT TOOL 1

**COURSE NAME: Artificial Intelligence
CSA17**

COURSE CODE:

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.*
(BTL 5)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Annexure A

**Duration : 1 Hour
Total Marks:50**

Design Task

Code of Conduct:

*I **BHUVANESHWARAN J(192511402)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: *Bhuvaneshwaran J*

Design Task

AI-Based Network Intrusion Detection Framework

Design a scalable intrusion detection system using PySpark RDDs to process large volumes of network traffic logs

in real time. Explain how anomaly detection algorithms can be implemented in a distributed manner across a cluster,

and how intelligent agents collaborate to identify suspicious traffic patterns, trigger security alerts, and continuously

update detection rules based on newly observed attack signatures.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations /actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined
Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality

Clarity, Presentation & Documentation	Very clear, neat diagrams, well- organized report	Mostly clear	Somewhat organized	Poor presentation
---	---	--------------	-----------------------	-------------------

ARTIFICIAL INTELLIGENCE – ASSESSMENT TOOL 1

Answers for Bhuvaneshwaran J | Register No.: 192511402

Course Code: CSA17 | CO5: Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures

Design Task

AI-Based Network Intrusion Detection Framework

Design a scalable intrusion detection system using PySpark RDDs to process large volumes of network traffic logs in real time. The system performs distributed anomaly detection across a cluster. Intelligent agents collaborate to identify suspicious traffic, trigger security alerts, and continuously update detection rules using newly observed attack signatures.

1. Problem Understanding and Objectives

The proposed system detects malicious or abnormal network activity from large-scale traffic logs. Traditional single-machine intrusion detection can become slow when traffic volume increases. The objective is to build a distributed framework that processes traffic in parallel and provides fast detection.

- Collect network traffic logs from routers, firewalls, servers, and intrusion sensors.
- Clean and transform raw logs into useful features such as source IP, destination IP, port, protocol, packet count, byte count, and connection duration.
- Use distributed anomaly detection to identify unusual traffic patterns.
- Use intelligent agents to analyze alerts, assign severity, and coordinate responses.
- Continuously improve detection rules from newly confirmed attack signatures.

2. Proposed System Architecture

The framework consists of data collection, distributed processing, anomaly detection, intelligent-agent coordination, and alert/response modules.

Layer	Function
Network Sources	Routers, firewalls, servers, IDS sensors and application logs
Data Ingestion	Collects and forwards high-volume network logs
PySpark Cluster	Distributes traffic records among worker nodes
RDD Processing	Filtering, mapping, grouping and feature extraction
Anomaly Detection	Calculates deviations from normal traffic behaviour
Intelligent Agents	Correlate events, assess risk and coordinate response
Alert & Response	Security alerts, blocking/quarantine and analyst notification
Model/Rule Update	Adds validated attack signatures and improves future detection

3. Data Flow and Processing Pipeline

Network traffic logs → ingestion layer → RDD creation → preprocessing → feature extraction → distributed anomaly detection → agent-based correlation → risk scoring → alert/response → feedback and rule update.

Example processing stages:

- **map()**: Convert each raw log entry into a structured record containing relevant network features.
- **filter()**: Remove invalid records and focus on relevant protocols, ports, IP ranges, or suspicious events.
- **map()** / feature extraction: Calculate traffic statistics for each connection or source.

- **groupByKey()** or **reduceByKey()**: Aggregate activity by source IP, destination IP, session, or time window.
- **mapValues()**: Calculate anomaly scores or risk-related statistics.
- **filter()**: Select records whose anomaly score exceeds the chosen threshold.
- **collect()** / controlled output: Send only required alert information to the alerting layer.

4. PySpark RDD Operations

RDDs provide a distributed and fault-tolerant representation of the traffic data. The same processing logic can execute in parallel on multiple worker nodes.

Illustrative PySpark design:

```
logs = sc.textFile("network_logs")
records = logs.map(parse_log)
valid = records.filter(is_valid)
by_source = valid.map(lambda x: (x.src_ip, x)).groupByKey()
scores = by_source.mapValues(calculate_anomaly_score)
alerts = scores.filter(lambda x: x[1] > THRESHOLD)
```

The example shows the intended distributed pipeline rather than a complete production IDS implementation.

5. Distributed Anomaly Detection

Each worker processes a partition of network records. Features are calculated locally where possible, while aggregation operations combine information across partitions. A traffic source can be considered suspicious when its behaviour significantly differs from a learned or configured normal baseline.

- High connection frequency from one source.
- Unexpected ports or protocols.
- Abnormally high packet or byte volume.
- Repeated failed connection attempts.
- Sudden changes from a normal traffic profile.
- Known malicious signatures appearing in traffic logs.

6. Intelligent Agent Integration

Intelligent agents act as autonomous decision-making components around the detection pipeline. Different agents can specialize in traffic analysis, alert correlation, response, and rule updating.

Agent	Responsibility
Traffic Analysis Agent	Examines traffic features and anomaly scores.
Correlation Agent	Combines related alerts from multiple sources.
Risk Assessment Agent	Assigns severity such as low, medium, high, or critical.
Response Agent	Triggers actions such as alerting, blocking, or isolating suspicious sources according to policy.
Learning/Rule Agent	Uses validated new attack signatures to update detection rules.

7. Alert and Response Process

When suspicious activity is detected, the analysis agent forwards the event to the correlation and risk agents. If the risk exceeds a policy threshold, the response agent generates a security alert and may request actions such as blocking an IP address or isolating a host. Confirmed incidents are fed back to the learning/rule agent so that future traffic can be detected more accurately.

8. Scalability and Distributed Computing

- **Parallelism:** Network logs are partitioned and processed simultaneously by multiple worker nodes.
- **Scalability:** Additional worker nodes can be added as traffic volume grows.
- **Fault tolerance:** RDD lineage allows lost partitions to be recomputed when required.
- **Load distribution:** Traffic records are divided across partitions to reduce processing time.
- **Real-time capability:** The architecture can be paired with a streaming ingestion layer so newly arriving traffic is processed continuously.

9. Continuous Rule and Model Updating

Newly observed attack signatures are first validated to reduce false positives. The learning/rule agent then updates the detection knowledge base. Historical traffic can be reprocessed in the PySpark cluster to evaluate the updated rules or retrain a detection model. This creates a continuous feedback loop: **Detect** → **Validate** → **Update** → **Reprocess/Evaluate** → **Improve**.

10. Advantages

- Handles large volumes of network traffic logs efficiently.
- Provides parallel processing through the PySpark cluster.
- Supports scalable anomaly detection.
- Allows autonomous agent-based alert coordination.
- Can continuously incorporate newly discovered attack signatures.
- Improves security monitoring speed and coverage.

11. Conclusion

The AI-Based Network Intrusion Detection Framework combines PySpark RDD-based distributed processing with intelligent agents to detect suspicious network behaviour at scale. RDD transformations and actions provide the processing foundation, while agents perform event correlation, risk assessment, response coordination, and continuous rule improvement. The resulting architecture is scalable, fault-tolerant, and suitable for large network environments.

Source basis: This answer is prepared specifically for Bhuvaneshwaran J (Register No. 192511402), whose assigned design task is the AI-Based Network Intrusion Detection Framework in the uploaded assessment document. The assessment specifies PySpark RDD processing, distributed anomaly detection, intelligent-agent collaboration, alerts, and continuous rule updates as the required focus.